

SAFETY HELMET DETECTION USING YOLOv8

Internship Information

Intern Name: Abhishek M Nair

Intern ID: CITS4346

Organization: CODTECH IT Solutions Private Limited

Domain: Artificial Intelligence

Internship Duration: 8 Weeks

1. Introduction

Detection of safety helmet is an essential application of computer vision that aims to make workers safe within an industrial setting. Monitoring by humans proves inefficient and time-consuming. Object detection methods based on deep learning have the ability to detect helmets from visual data.

This experiment focuses on detecting helmets using YOLOv8 through a custom-made dataset.

2. Objective

The objectives of this project are:

- Detect safety helmets automatically.
 - Train a custom YOLOv8 model.
 - Perform object detection on test images.
 - Analyze model performance.
 - Generate confidence and accuracy metrics.
 - Visualize training and evaluation results.
-

3. Problem Statement

Industrial safety is a major concern in construction sites and factories. Workers without helmets are exposed to severe risks. Therefore, an intelligent detection system is required to automatically identify safety helmets and monitor compliance.

4. Technologies Used

Technology	Purpose
Python	Programming Language
YOLOv8	Object Detection
OpenCV	Image Processing
NumPy	Numerical Computation
Pandas	Data Analysis
Matplotlib	Visualization
Jupyter Notebook	Development Environment
VS Code	IDE

5. Dataset Description

The project uses a custom Safety Helmet Detection dataset.

Dataset Structure

```
train/
├── images/
└── labels/

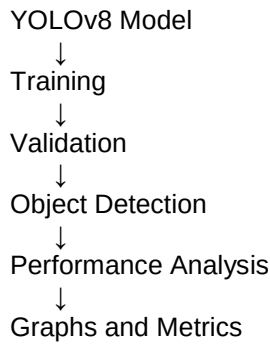
test/
├── images/
└── labels/
```

Classes

Class ID	Class Name
0	Helmet
1	Head
2	Person

6. Project Architecture

```
Input Images
  ↓
Dataset Preparation
  ↓
```



7. Methodology

Step 1: Dataset Preparation

Training and testing images along with annotation files were collected and organized.

Step 2: Data Analysis

The distribution of object classes was analyzed to understand the dataset characteristics.

Step 3: Model Training

The YOLOv8 Nano model was fine-tuned using the custom dataset.

Training Parameters

Parameter	Value
Epochs	50
Batch Size	8
Image Size	640
Optimizer	AdamW
Learning Rate	0.001

Step 4: Model Validation

The model was evaluated using:

- Precision
- Recall
- mAP50
- mAP50-95

Step 5: Object Detection

The trained model predicts helmets in test images and generates bounding boxes with confidence scores.

Step 6: Confidence Analysis

Confidence values of detected objects were analyzed and average confidence graphs were generated.

Step 7: Performance Visualization

Performance graphs were generated for:

- Training Loss
- mAP
- Precision
- Recall

8. Folder Structure

```
Safety-Helmet-Detection-YOLOv8
├── dataset
│   └── safety-helmet-dataset.yaml
├── train
│   ├── images
│   └── labels
├── test
│   ├── images
│   └── labels
├── notebooks
│   └── helmet_detection_yolov8.ipynb
├── outputs
├── runs
├── screenshots
├── documentation
├── README.md
└── requirements.txt
```

9. Results

The model successfully detected helmets and achieved satisfactory performance.

The following outputs were generated:

- Dataset Count Output
 - Class Distribution Graph
 - Sample Training Images
 - Training Completed Output
 - Validation Metrics
 - Detection Output
 - Object Count Graph
 - Average Confidence Graph
 - Training Loss Graph
 - mAP Graph
 - Precision and Recall Graph
-

10. Performance Metrics

The model evaluation includes:

- Precision
 - Recall
 - mAP50
 - mAP50-95
 - Box Loss
 - Classification Loss
 - DFL Loss
-

11. Applications

- Construction Site Monitoring
 - Industrial Safety Systems
 - Worker Safety Compliance
 - Smart Surveillance Systems
 - Manufacturing Industries
-

12. Future Enhancements

- Real-time Webcam Detection
 - Video-based Helmet Detection
 - Streamlit Web Application
 - Edge Device Deployment
 - Industrial Monitoring Systems
 - CCTV-based Safety Monitoring
-

13. Conclusion

This project demonstrates the use of deep learning and computer vision techniques for automatic safety helmet detection. YOLOv8 was successfully trained on a custom dataset and achieved reliable performance. The generated evaluation metrics and graphs indicate that the system can be extended for real-time industrial safety applications.

References

1. Ultralytics YOLOv8 Documentation
 2. OpenCV Documentation
 3. Python Documentation
 4. Pandas Documentation
 5. Matplotlib Documentation
 6. Roboflow Dataset Repository
-

Author

Abhishek M Nair
Artificial Intelligence Intern
CODTECH IT Solutions Private Limited
Intern ID: CITS4346
